

B. Chaithanya Prakash Reddy

REG NO: 192425255

COURSE: CSA0603 DESIGN AND DATA ANALYSIS FOR ALGORITHM

CO5_AT1_CONSTRAINT BASED PROBLEM SOLVING

Q6. N-Queens Problem – Backtracking / Constraint Satisfaction Problem

Problem Statement

A chessboard of size $N \times N$ must be filled with N queens such that no two queens attack each other. The solution must clearly define the row, column, and diagonal constraints, explain how these constraints reduce the search space, and develop a backtracking-based method that places queens step-by-step while validating constraints. Pruning techniques are applied to eliminate invalid placements as early as possible. The solution must also justify completeness and discuss computational complexity.

1. Problem Definition

The N-Queens problem is a classical constraint satisfaction and backtracking problem. The objective is to place N queens on an $N \times N$ chessboard so that no two queens can attack one another. Since a queen can attack another queen horizontally, vertically, or diagonally, every placement must satisfy all of these restrictions.

The problem can be represented by assigning exactly one queen to each row. Once a row is selected, the algorithm tries possible columns and checks whether the chosen square is safe with respect to queens already placed in earlier rows. If the position is unsafe, it is rejected immediately. If no column is safe for the current row, the algorithm backtracks to the previous row and changes that queen's position.

For an $N \times N$ board:

- There are N rows and N columns.
- Exactly N queens must be placed.
- Each row contains exactly one queen.
- Each column contains at most one queen.
- No two queens can share a diagonal.

By placing one queen per row, the search is organized row-by-row. This removes the need to consider multiple queens in the same row and makes the recursive state compact.

2. Constraints

The important constraints in the N-Queens problem are:

2.1 Row Constraint

No two queens may occupy the same row. The backtracking algorithm satisfies this constraint naturally by placing exactly one queen when processing each row. The recursion moves from row 0 to row 1, then row 2, and so on, so a row is never assigned more than once.

2.2 Column Constraint

No two queens may occupy the same column. If a queen has already been placed in column c , another queen cannot be placed in column c . A boolean column array can record whether each column is occupied.

2.3 Main Diagonal Constraint

Two squares lie on the same main diagonal when their row and column differences are equal. For a cell (r, c) , the value $r - c$ uniquely identifies its main diagonal. Therefore, a placement is invalid if another queen already occupies the same $r - c$ diagonal.

2.4 Secondary Diagonal Constraint

Two squares lie on the same secondary diagonal when their row and column sums are equal. For a cell (r, c) , the value $r + c$ identifies its secondary diagonal. Therefore, a placement is invalid if another queen already occupies the same $r + c$ diagonal.

The complete safety condition for placing a queen at (r, c) is:

```
column[c] == false
mainDiagonal[r - c] == false
secondaryDiagonal[r + c] == false
```

Only when all three tests are satisfied is the square considered safe. The row constraint is already guaranteed by the recursive construction.

3. Constraint Representation

A convenient representation stores the board using arrays rather than repeatedly scanning every square. The following data structures can be maintained during the search:

- `column[c]` – true if column c already contains a queen.
- `mainDiagonal[r - c]` – true if the corresponding main diagonal is occupied.
- `secondaryDiagonal[r + c]` – true if the corresponding secondary diagonal is occupied.

Because $r - c$ can be negative, an offset of $N-1$ can be added when an array implementation is used. Thus, the main diagonal index can be stored as $r - c + (N - 1)$. The secondary diagonal index remains $r + c$.

For an $N \times N$ board there are $2N-1$ diagonals of each direction. Hence, the arrays can be sized as:

```
column:           N entries
mainDiagonal:     2N - 1 entries
secondaryDiagonal: 2N - 1 entries
```

The board itself can also be stored as a matrix for displaying the final configuration. However, the three constraint arrays are sufficient for efficient validation during backtracking.

4. How the Constraints Reduce the Search Space

Without constraints, selecting N positions from an $N \times N$ board would create a very large search space. If queens were placed independently, each queen could initially appear in any of N^2 cells. Constraint-based backtracking avoids exploring most of these possibilities.

First, the row constraint reduces the construction to one choice per row. Therefore, instead of selecting arbitrary board cells, the algorithm considers a sequence of N column choices, one for each row.

The column constraint further reduces the choices because a column already occupied by a queen is removed from consideration. After k queens have been placed, k columns are unavailable.

The diagonal constraints remove additional positions. A candidate square that lies on either diagonal of an existing queen is rejected immediately. This prevents the algorithm from building a partial solution that is already impossible.

This early rejection is called pruning. Pruning is important because a single invalid queen can make many future arrangements impossible. Rejecting the branch at the first detected conflict prevents the algorithm from exploring all of those descendants.

5. Backtracking Solution

Backtracking constructs a solution incrementally. At each recursive level, one row is processed. Every column in that row is tested. If the square is safe, the queen is placed and recursion continues. If the deeper search fails, the queen is removed and the next column is tried.

Step-by-Step Procedure

1. Start with row 0 and an empty board.
2. Try each column in the current row.
3. Check the column, main diagonal, and secondary diagonal constraints.
4. If the position is safe, place a queen and mark the three constraint structures.
5. Move to the next row recursively.
6. If all N rows are successfully processed, a valid configuration has been found.
7. If the current row has no safe column, return to the previous row.
8. Remove the previous queen, clear its constraint marks, and try the next available column.
9. Continue until a solution is found or all possibilities have been exhausted.

Pseudocode

```
NQueens(row) :  
    if row == N:  
        record solution  
        return  
  
    for col = 0 to N-1:  
        d1 = row - col + (N - 1)  
        d2 = row + col
```

```

if column[col] == false
  and mainDiagonal[d1] == false
  and secondaryDiagonal[d2] == false:

    board[row][col] = 1
    column[col] = true
    mainDiagonal[d1] = true
    secondaryDiagonal[d2] = true

    NQueens(row + 1)

    board[row][col] = 0
    column[col] = false
    mainDiagonal[d1] = false
    secondaryDiagonal[d2] = false

```

6. Pruning Techniques

Pruning means stopping a search branch as soon as it violates a constraint. In the N-Queens problem, pruning is built directly into the safety test.

6.1 Column Pruning

If `column[c]` is already true, the candidate (row, c) is rejected. There is no reason to place another queen in that column because the two queens would attack each other vertically.

6.2 Diagonal Pruning

If either diagonal array indicates that the candidate square shares a diagonal with an existing queen, the candidate is rejected immediately. This avoids creating partial boards that can never become valid.

6.3 Immediate Failure Detection

When a row has no legal column, the algorithm does not proceed to later rows. It immediately backtracks. For example, if the fourth row has no safe position, there is no reason to construct a fifth, sixth, or later row from that partial configuration.

6.4 Symmetry Reduction

An optional optimization is to exploit symmetry. Reflected or rotated solutions can be related to one another. For example, when only one representative from symmetric starting positions is required, the first-row search can sometimes be restricted. This can reduce duplicated work, although the basic backtracking algorithm does not require symmetry handling.

7. Example: 4-Queens

For $N = 4$, the algorithm searches for four queens on a 4×4 board. One valid configuration is represented by the column sequence `[1, 3, 0, 2]`, where the number at each position indicates the queen's column for that row.

```

. Q . .
. . . Q
Q . . .
. . Q .

```

For row 0, column 1 is selected. In row 1, columns that conflict with the first queen are rejected. The algorithm continues this process. Whenever a selected position creates a column or diagonal conflict, that branch is pruned. Eventually the sequence [1, 3, 0, 2] satisfies all constraints.

The important point is that the algorithm does not blindly enumerate every 4×4 board. It rejects invalid placements while the partial configuration is still small.

8. Constraint Satisfaction Problem (CSP) Formulation

The N-Queens problem can also be formulated as a Constraint Satisfaction Problem. Each row is treated as a variable, and its domain consists of the N possible columns.

- Variables: $Q_1, Q_2, \dots, Q_N$, one variable for each row.
- Domain: $\{1, 2, \dots, N\}$, representing the possible column positions.
- Column constraint: $Q_i \neq Q_j$ for $i \neq j$.
- Diagonal constraint: $|Q_i - Q_j| \neq |i - j|$ for $i \neq j$.

The diagonal condition states that two queens cannot have a column difference equal to their row difference. Together, the column and diagonal constraints guarantee that no two queens attack each other.

9. Correctness of the Backtracking Algorithm

The algorithm is correct because every queen is placed only after checking all relevant constraints against the queens already placed. Since one queen is placed per row, row conflicts cannot occur.

The column array guarantees that two queens never occupy the same column. The main-diagonal and secondary-diagonal arrays guarantee that no two queens occupy the same diagonal. Therefore, every complete board produced by the algorithm is a valid N-Queens configuration.

The algorithm is also complete. For each row, it considers every column that has not been pruned by the constraints. If a partial choice cannot lead to a solution, recursion eventually reaches a row with no valid position and backtracks. The algorithm then tries another choice. Consequently, every non-pruned candidate configuration is explored.

If the task is to find one solution, the search can stop when the first complete configuration is obtained. If the task is to find all solutions, the algorithm continues after recording each complete configuration until every branch has been considered.

10. Why All Valid Configurations Are Ensured

Completeness follows from the systematic enumeration of choices. At each row, the algorithm loops through all columns from 0 to $N-1$. A column is skipped only when it violates a constraint with an

already placed queen. A valid final configuration can never contain such a violating placement, so pruning cannot remove a genuinely valid solution.

Backtracking restores the state after every failed choice. The board cell is cleared and the corresponding column and diagonal markers are reset. This restoration ensures that an earlier choice does not incorrectly affect later branches.

11. Computational Complexity

The N-Queens problem has an exponential search space. With one queen fixed in every row, a simple upper-bound view is $O(N^N)$, because each of N rows could initially choose one of N columns. Enforcing the column constraint reduces the unpruned arrangement count to at most $N!$, before diagonal pruning is considered.

In practice, diagonal and column pruning substantially reduce the number of states visited, but the worst-case time complexity remains exponential. For a backtracking solver, a commonly stated practical upper bound is $O(N!)$, while the exact running time depends on the pruning strategy and whether one or all solutions are required.

Space complexity is $O(N)$ for the recursion stack, column array, and the two diagonal arrays, excluding the optional $N \times N$ board used for display. If the full board matrix is maintained, the storage becomes $O(N^2)$. If all solutions are stored, additional output space is required.

12. Comparison: Naive Search vs Pruned Backtracking

The effect of the constraints can be summarized as follows:

Approach	Validation	Search Effect
Naive placement	Checks complete arrangements late	Many invalid boards explored
Row-wise placement	One queen per row	Removes same-row cases
Column pruning	Rejects occupied columns	Removes vertical conflicts early
Diagonal pruning	Rejects both diagonal conflicts	Removes large invalid branches
Backtracking	Stops failed branches immediately	Avoids exploring impossible descendants

Thus, the constraints are not only rules for checking the final answer; they are also tools for controlling the search process. The earlier an invalid state is detected, the more unnecessary computation is avoided.

13. Feasibility Conditions

A final N-Queens configuration is feasible only when:

- Exactly N queens are placed.
- Every row contains exactly one queen.
- No two queens occupy the same column.
- No two queens share a main diagonal.
- No two queens share a secondary diagonal.
- All placements remain consistent with the constraints throughout the construction.

Conclusion

The N-Queens problem is a clear example of constraint-based problem solving using backtracking. The board is constructed one row at a time, and every candidate position is checked against row, column, and diagonal constraints. Column and diagonal arrays allow these checks to be performed efficiently.

Pruning eliminates unsafe positions before they can generate larger invalid search branches. Backtracking then reverses failed decisions and systematically tries alternatives. Because only constraint-violating branches are discarded, every valid configuration remains reachable. Therefore, the method can find one solution or enumerate all solutions while using significantly less search than a naive brute-force approach.

Overall, the combination of constraint representation, early pruning, and systematic backtracking provides a correct and effective solution to the N-Queens problem and demonstrates the principles of Constraint Satisfaction Problems and exponential search.